Optimization Techniques for Digital VLSI Design

Module 5: Verification

Santosh Biswas

IIT Guwahati

Course Plan

Module 5: Verification

Lecture 1: LTL/CTL based Verification

Lecture 2 and 3: Verification of Large Scale Systems

- BDD based verification
- ADD, HDD based verification

Lecture 4, 5: Symbolic Model Checking

Lecture 6: Bounded Model Checking

Binary Decision Tree and Circuits

Binary decision tree (BDT) for a Boolean function is a tree such that

- The internal nodes of a tree are labeled by variables of the function.
- The leaves of a tree are labeled by 0 and 1.
- Every internal node in a tree has exactly two children, the two arcs from node to the children are labeled by 0 (dashed line) and by 1 (solid line). The dashed line indicates that the value assigned to the variable is 0 and the solid line indicates that the value assigned to the variable is 1.

Representing Circuits

- A circuit having n inputs as $i_1, i_2, i_3 \ldots i_n$ and m outputs as $o_1, o_2, o_3 \ldots o_m$ can be represented in many ways which can be directly used for equivalence checking (or verification).

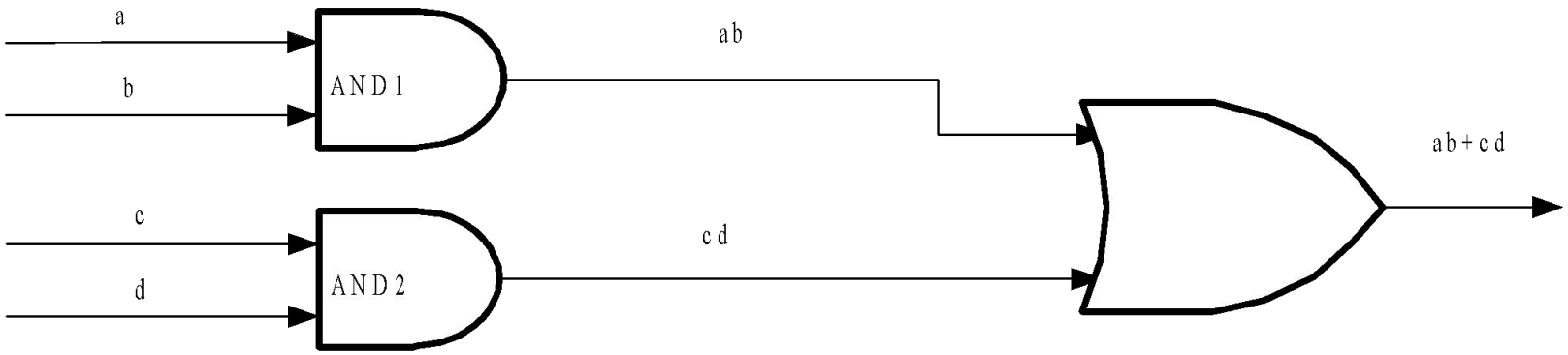
- The simplest way is to use truth tables. The truth table comprises 2^n rows starting from $i_1 = 0, i_2 = 0, i_3 = 0 \ldots i_n = 0$ to $i_1 = 1, i_2 = 1, i_3 = 1 \ldots i_n = 1$ (each row corresponds to an input combination).

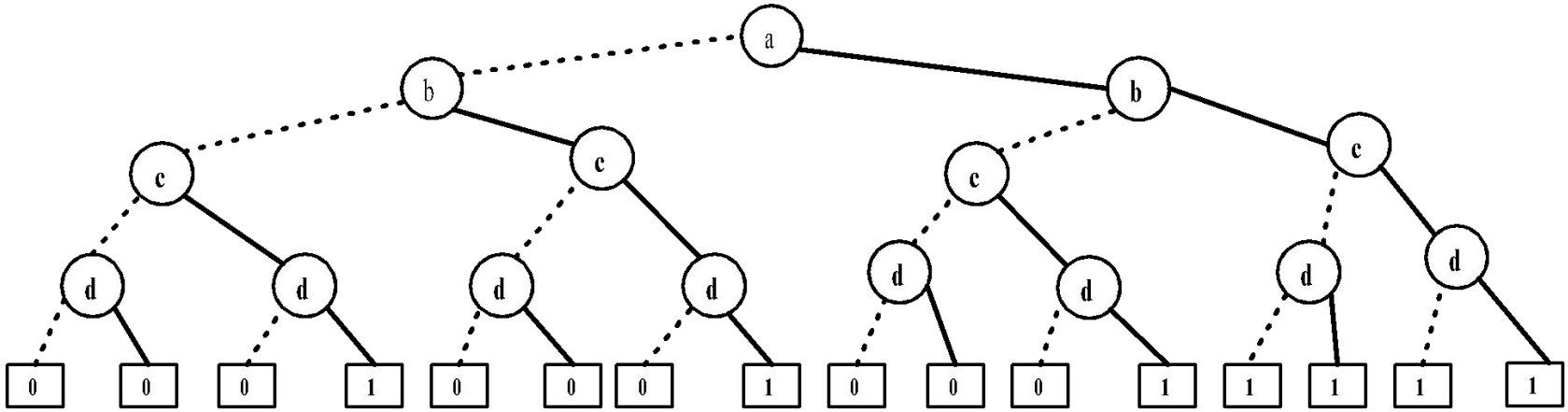
- Also output values of $o_1, o_2, o_3 \ldots o_m$ for the corresponding input combinations are stored.

Binary Decision Tree and Circuits

The following Boolean function f on four variables *a, b, c* and *d*.

$$f(a,b,c,d) = ab + cd$$





Binary Decision Tree and Circuits

A circuit having n inputs and m outputs has $O(2^n \times m)$ entries in the truth table; 2^n rows and m outputs for each row.

Similarity, for such a circuit the number of nodes in the BDT is $O((2^{n-1} + 2^n) \times m)$

Binary Decision Diagrams

From the example illustrated in the last section, it may be noted that BDT cannot be generated within practical time lines for a reasonably complex circuit. The main reason is the exponential number of nodes in a BDT with respect to number of inputs. However, it may be noted that there are many redundant nodes in the BDT; simply, in the leaf level we can have one node with "0" and the other with "1" and redirect paths from the nodes of leaf level -1 accordingly. Similarly, this reduction is carried out at all layers till the root.

Broadly speaking, a Binary Decision Diagram (BDD) is a directed acyclic graph representation of the Boolean expression which cannot further be reduced my eliminating redundant nodes. In other words, just like BDT, BDD is an acyclic graph representation of a Boolean expression but unlike BDT, it does not comprise any redundant node either in the leaf or the non-leaf level. So BDD is called Reduced BDD (RBDD).

Binary Decision Diagrams

The reduction from BDT to RBDD is done using three rules.
R1: Removing of duplicate terminals (leaf nodes): If a BDD contains more than one terminal 0-node, then we redirect all edges which point to such 0-node to just one of them and other 0-node can be removed. Similarly we do with 1-node also.
R2: Removal of duplicate non terminals (internal nodes): If two distinct nodes *n* and *m* in the RBDD are the roots of structurally identical sub-BDDs, then we eliminate one of them, say *m*, and redirect all its incoming edges to the other one (node *n*)
R3: Removal of Redundant test: If both outgoing edges of a node *n* point to the same node *m*, then we eliminate the node *n*, and point all its incoming edges to *m*.

Binary Decision Diagrams

Removal of Duplicate Terminals (Leaf Nodes)

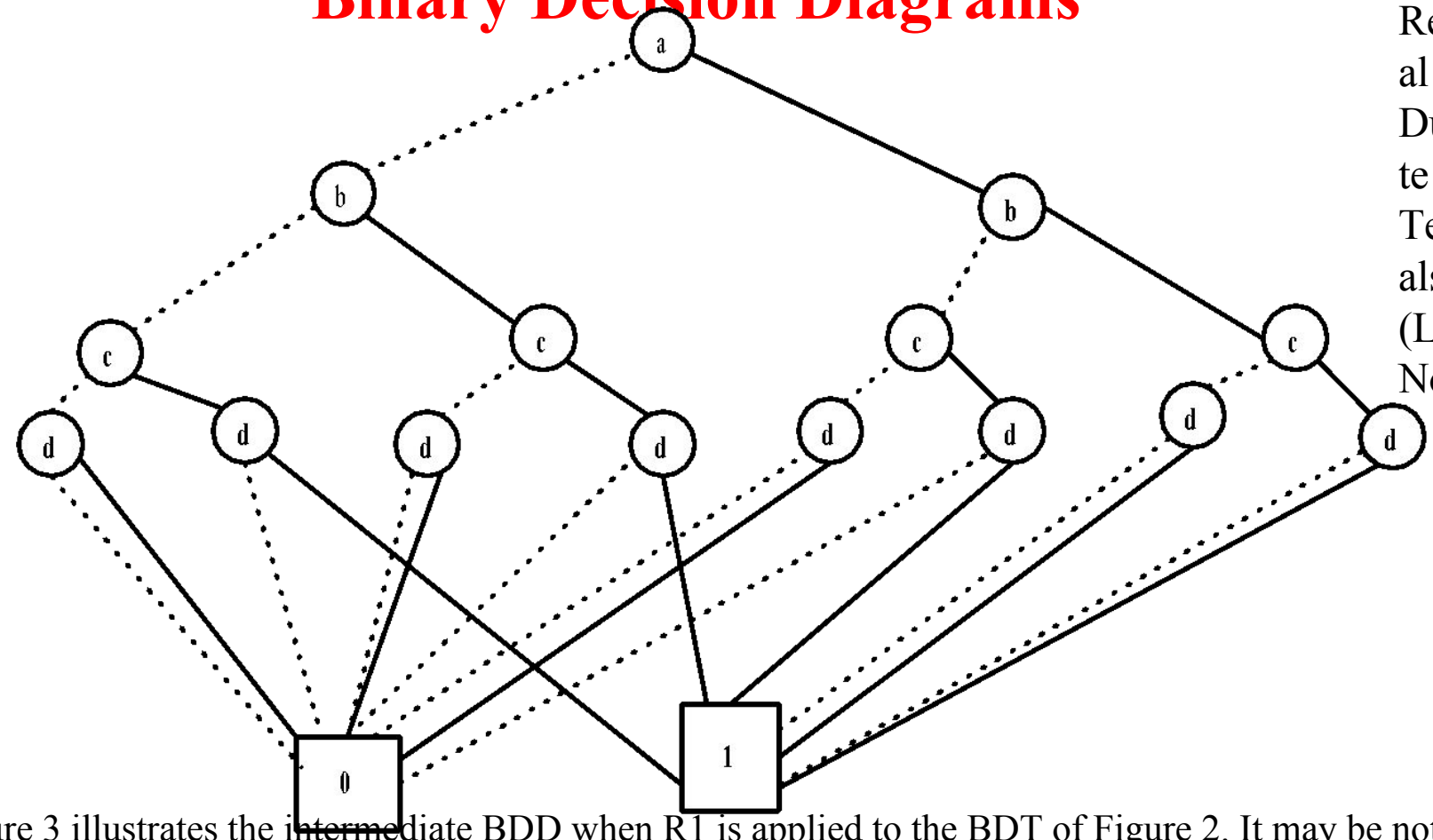
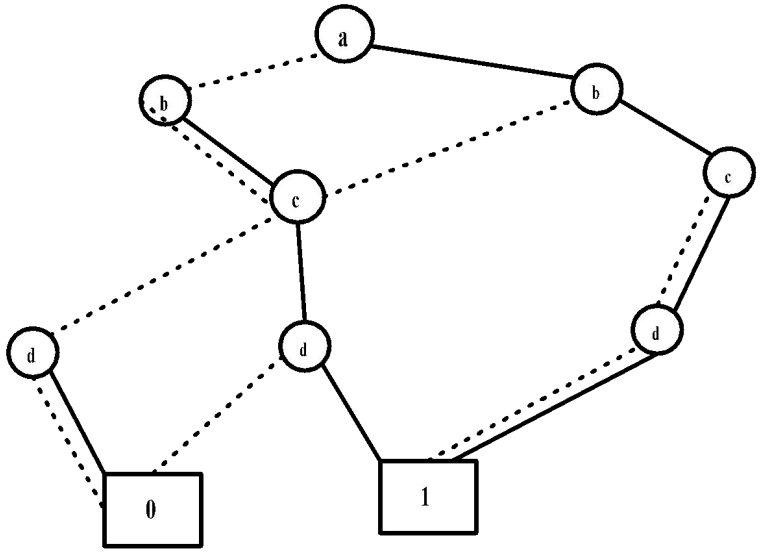


Figure 3 illustrates the intermediate BDD when R1 is applied to the BDT of Figure 2. It may be noted that there is only one leaf node with “0” and another one with “1”. From Figure 2 it may be observed that the path from root to leaf node corresponding reaches a leaf node with “0”. Also the path from corresponding to reaches a leaf node with “0”. Now as there is a single node at leaf level with “0” in the RBDD (Figure 3) both the paths corresponding to and has been redirected the single “0” leaf node. Similarly, redirection of all paths in Figure 3 can be explained.

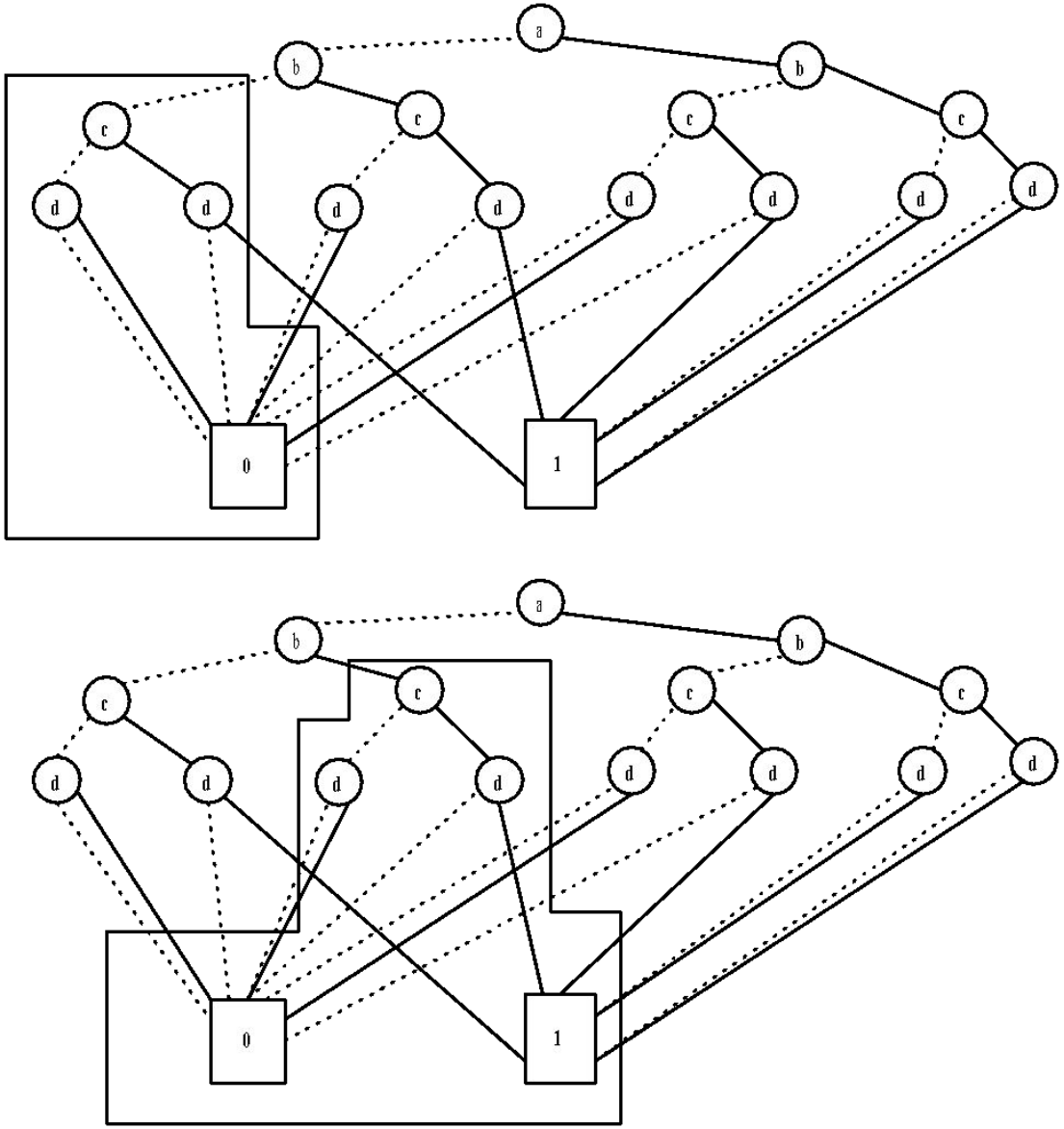
Binary Decision Diagrams



Removal of Duplicate non terminals

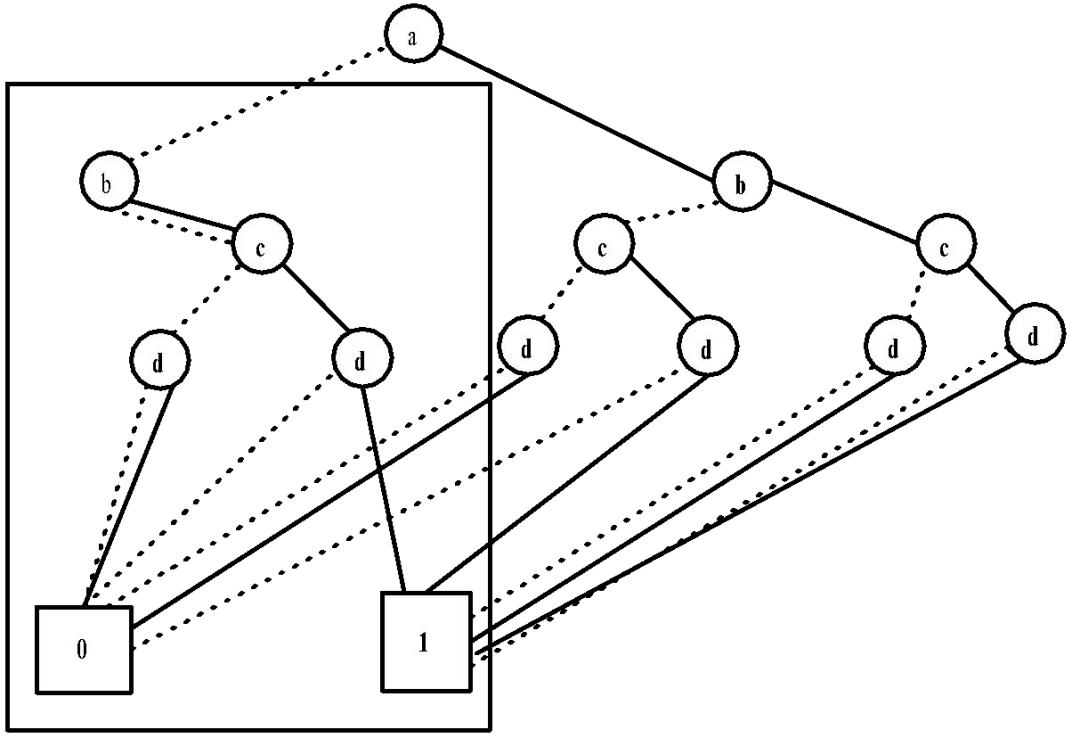
Figure 4 illustrates the intermediate BDD when R2 is applied to the (intermediate) BDD of Figure 3. From Figure 5 it may be noted that the sub-BDDs enclosed by polygons are structurally identical. So we retain only one of them and redirect edges incoming to the eliminated sub-BDD to the one being retained; resulting intermediate BDD is shown in Figure 6. When R2 is applied to all structurally identical sub-BDDs of Figure 3 we obtain the BDD of Figure 4.

Binary Decision Diagrams



A pair of duplicate sub-BDDs (enclosed by polygons) in the intermediate BDD of Figure 3

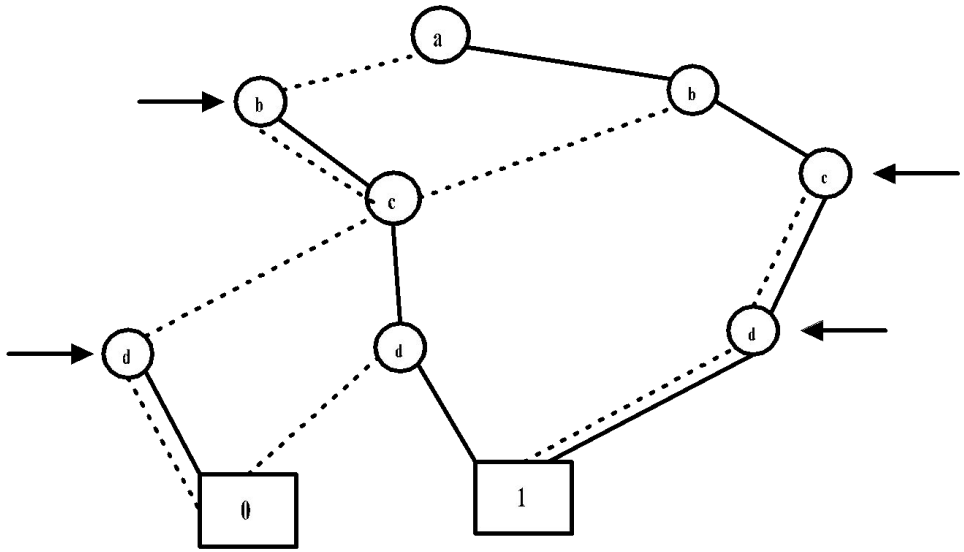
Binary Decision Diagrams



Intermediate BDD after applying R2 to the duplicate sub-BDDs shown in Figure 5

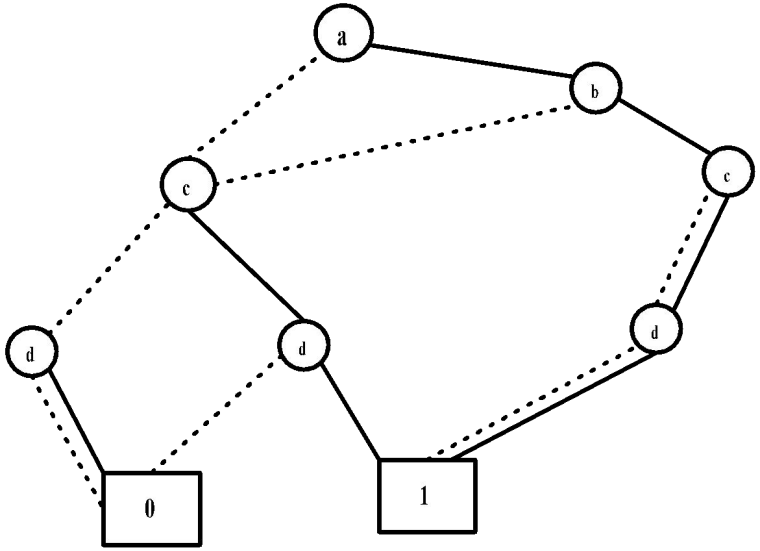
From Figure 4 it may be noted that there nodes *n* say, where both the outgoing edges point to the same node *m* say ; R3 eliminates such a node *n* and points all its incoming edges to *m*. Nodes of such type in BDD of Figure 4 are marked and shown in Figure 7. Figure 8 illustrates the intermediate BDD when R3 is applied to node "b" (marked by an arrow) whose both the outgoing edges point to the same node "c". When R3 is applied to all such nodes, the BDD obtained is shown in Figure 9.

Binary Decision Diagrams

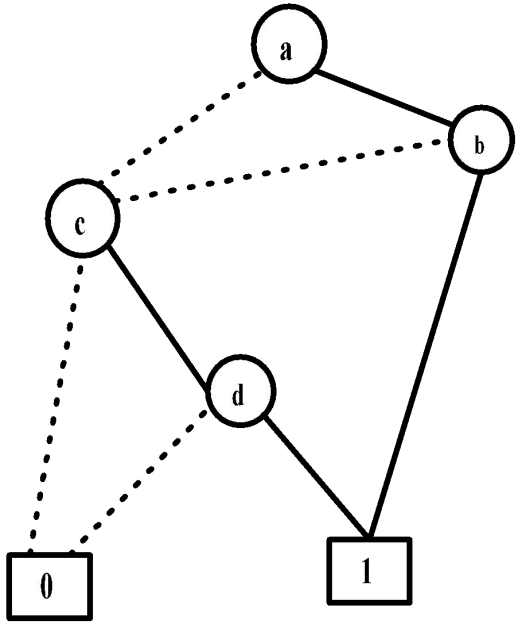


Nodes (arrow marked) where both the outgoing edges point to the same node

Binary Decision Diagrams



Intermediate BDD when R3 is applied to node "b" (Figure 7)



Removal of Redundant Test

RBDD using Shannon expansion

In the last section we demonstrated that a RBDD comprises much less number of nodes compared to its equivalent BDT. However, the construction started from a BDT and was finally transformed to a RBDD. As already discussed, time taken to generate a BDT is prohibitive for even a function with reasonable complexity. To obtain RBDD, it is not necessary to start with BDT at the first place. The discussion in the last section was only to illustrate that RBDD is a BDT without any redundancy and there are lots of redundant nodes in a BDT. Now we discuss how RBDD can be generated directly without use of a BDT.

RBDD is based on Shannon Expansion, according to which Boolean function can be represented by the sum of two sub-functions of the original. In particular, a function *f(x)* can be written as:

$f(x) = x.f[1/x] + x'.f[0/x]$, where x' means complement of x and *f[1/x]* and *f[0/x]* mean function obtained on replacing the variable x with 1 and 0 respectively.

For example, if $f(a) = ac + bc + ab$, then $f[1/a] = 1.c + bc + 1.b = b + c + bc$ and $f[0/a] = 0 + bc + 0$, so that finally *f(a)* can be written as: $f(a) = a.(b + c + bc) + a'(bc)$.

If we expand $a.(b + c + bc) + a'(bc)$, we get $ab + ac + abc + a'bc = ab + ac + bc.(a + a')$, which is $ab + bc + ac$ (*the form of f(a), we started with*).

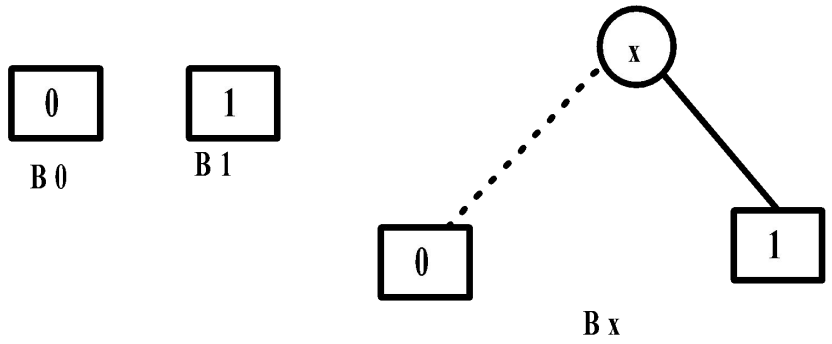
RBDD using Shannon expansion

The basic or primitive BDDs to represent Boolean function are given below.

B_0 : represents the Boolean constant 0.

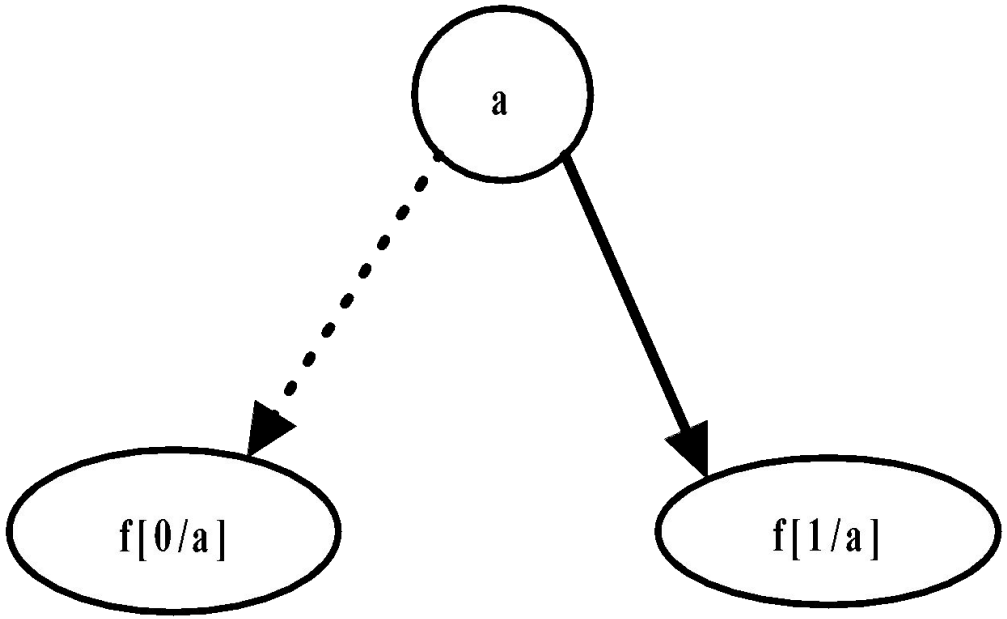
B_1 : represents the Boolean constant 1.

B_x : represents the Boolean variable x.



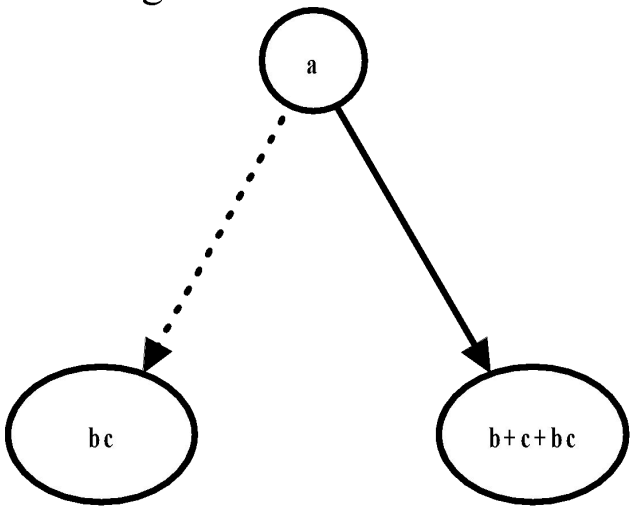
RBDD using Shannon expansion

Diagrammatically, a BDD of a function of variable *a* is rooted at a node representing variable *a*, and its children represent the two sub-functions whose sum results in the original expression. The child of the root with dotted line from parent represents *f[0/a]* and the one with solid line from parent represents *f[1/a]*. BDD of a function of variable *a* is shown in Figure 12.



RBDD using Shannon expansion

For instance, for the above considered example (i.e., $f(a) = ac + bc + ab$) the initial diagram would be of the form shown in Figure 13.



Further expansion of the sub-functions would lead to the following expressions, which are to be found out till we find the leaf node of the diagram (that is binary 0 or 1):

Let $f[1/a] = b + c + bc = g(b)$ and $f[0/a] = bc = h(b)$,

$g[1/b] = 1 + c + 1.c = 1$ and $g[0/b] = c = i(c)$

$h[1/b] = c = i(c)$ and $h[0/b] = 0$

$i[1/c] = 1$ and $i[0/c] = 0$

RBDD using Shannon expansion

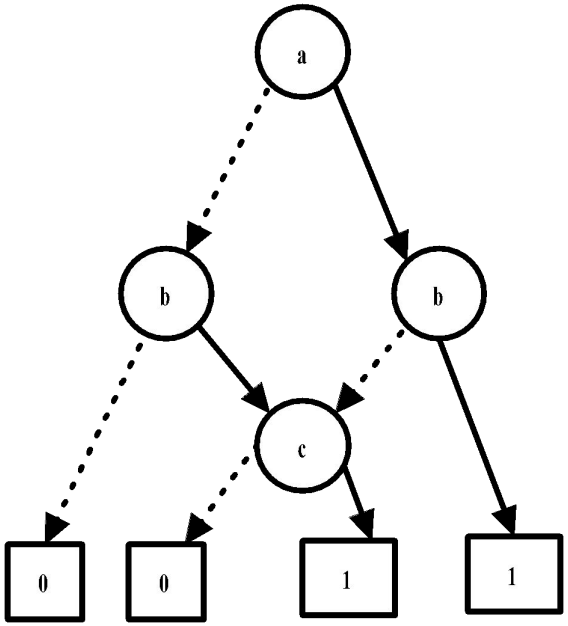
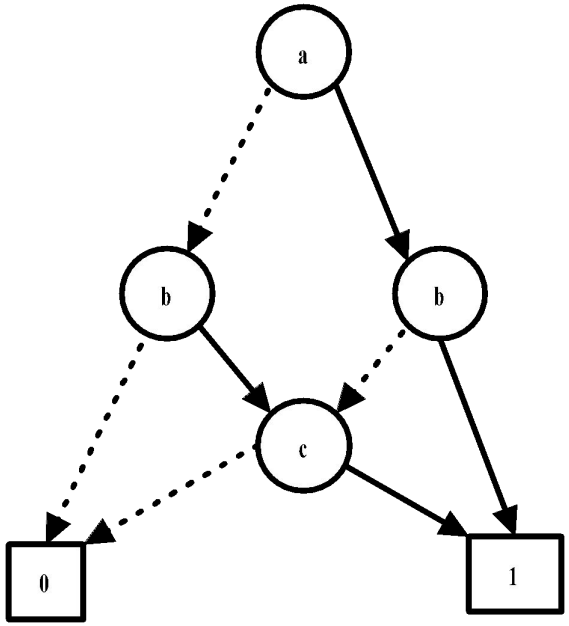


Diagram of the function $f(a) = ac + bc + ab$ with redundant nodes



Final diagram (RBDD) of the function $f(a) = ac + bc + ab$

Reduced Ordered BDD (ROBDD)

- In the previous lecture we have discussed how an RBDD represents a Boolean function, similar to truth table or Binary decision tree, however, the number of nodes do not grow exponentially with increase in number of inputs of the function. So RBDD is suitable for representing Boolean functions. It was also stated in the lecture that ordering of variables in the context of RBDD is important because
- Number of nodes for a given Boolean function depends on the ordering
- There may be inconsistent paths in the BDD if there is no ordering.
- In this lecture we will discuss Ordered BDD (which also called Ordered Reduced BDD).

Reduced Ordered BDD (ROBDD)

An ordered BDD (OBDD) is a BDD which has an ordering for the variables of the Boolean function it is representing. Let $[x_1, \ldots, x_n]$ be an ordered list of variables without duplications and let B be a BDD all of whose variables (of the corresponding Boolean function) occur somewhere in the list. We say that B has the ordering $[x_1, \ldots, x_n]$ if all variable labels of B occur in that list and, for every occurrence of x_i followed by x_j along any path in B, we have $i < j$. In other words, BDD B has the ordering $[x_1, \ldots, x_n]$, if the root node corresponds to variable x_1, nodes of level 2 correspond to x_2 and so on, till the nodes of level leaf-1 correspond to x_n. The BDD *B1* of Figure 1 has the variable ordering *[*X_1*,* X_2*,* X_3*,* X_4*,* X_5*]* and the BDD *B2* of Figure 2 has the variable ordering *[*X_5*,* X_4*,* X_3*,* X_2*,* X_1*]*.

Reduced Ordered BDD (ROBDD)

$$f(x1,\ x2,\ x3,\ x4,\ x5,\ x6) = (x1 + x2).(x3 + x4).(x5 + x6)$$

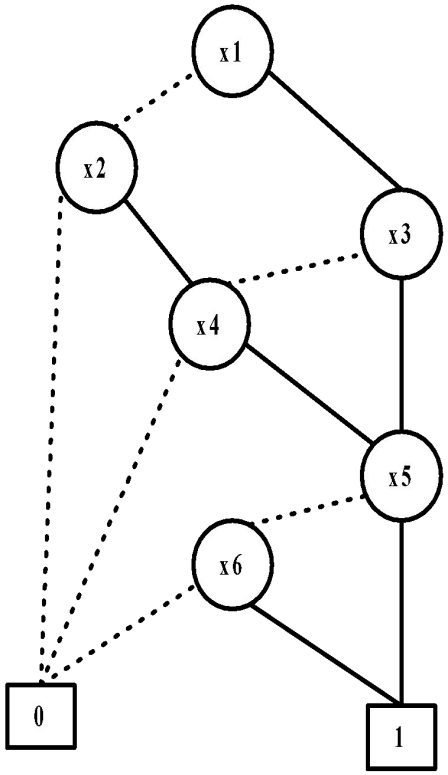


Figure 8. ROBDD of function f with variable ordering $[x1, x2, x3, x4, x5, x6]$

Reduced Ordered BDD (ROBDD)

$$f(x1,\ x2,\ x3,\ x4,\ x5,\ x6) = (x1 + x2).(x3 + x4).(x5 + x6)$$

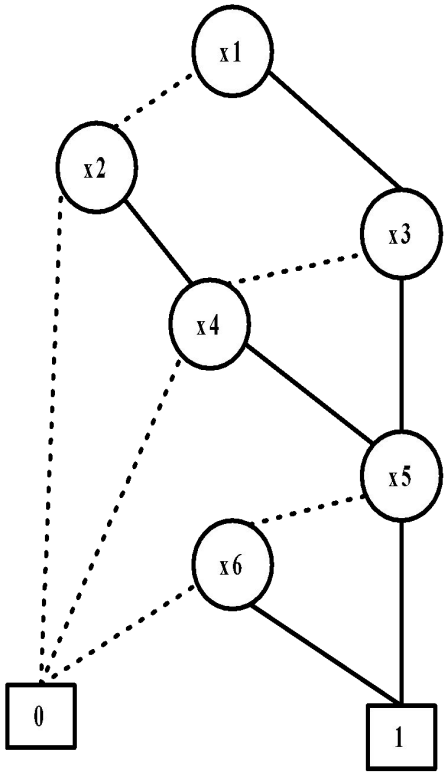
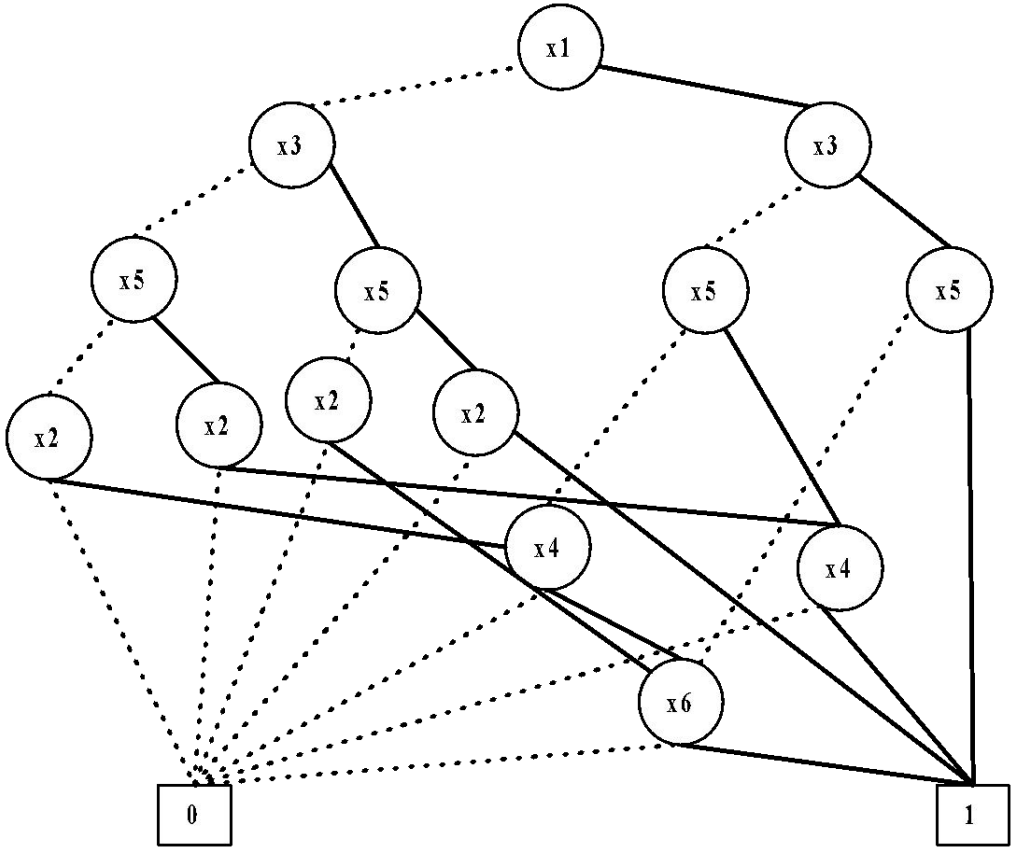


Figure 8. ROBDD of function f with variable ordering $[x1, x2, x3, x4, x5, x6]$

Reduced Ordered BDD (ROBDD)

$$f(x1,\ x2,\ x3,\ x4,\ x5,\ x6) = (x1 + x2).(x3 + x4).(x5 + x6)$$



[x1, x3, x5, x2, x4, x6]

Figure 9. ROBDD of function *f* with variable ordering

Applications of ROBDD

Expression Equivalence: Instead of converting expressions to be compared to SOP, POS, truth table or DBT form to test equivalence, we can construct ROBDD for the expressions using a common order. If the ROBDD's obtained are same then expressions are equivalent.

Validity of Boolean Expression: A Boolean expression is valid if it returns TRUE for all possible variable assignments. This can be tested by noticing occurrence of terminal node 0 in the ROBDD. If there is no 0 in the ROBDD then the expression is valid, else not.

Satisfiability of Boolean Expression: A Boolean expression is satisfiable if it returns TRUE for some variable assignment. This can be tested by noticing occurrence of terminal node 1 in the ROBDD. If there is a 1 in the ROBDD then expression is satisfiable, else not

.

Absence of Redundant Variable: In case there is a redundant variable in the expression, it can be found out by constructing ROBDD of the expression. If the diagram does not have any occurrence of a particular variable, then the variable is redundant to the expression.

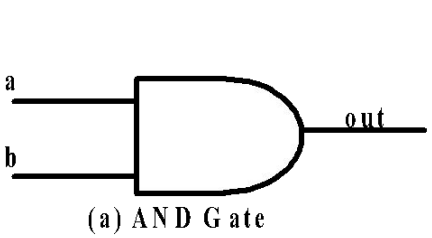
Operations on OBDD

Operations on ROBDD

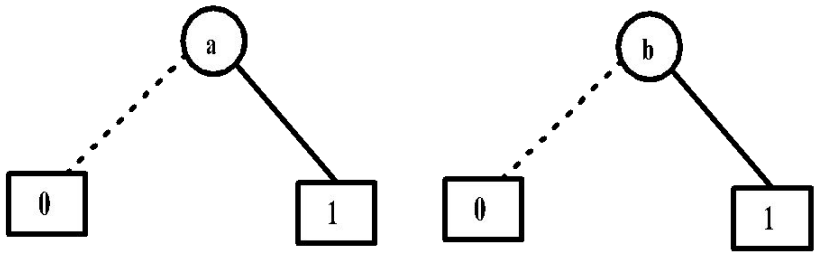
- The ROBDD for the output of the AND gate is shown below, which is computed using AND operation on the ROBDDs of the two inputs.

- ROBDD for the AND gate (or any other circuit) can be constructed by first determining the Boolean function at output and then constructing the ROBDD using Shannon's expansion.

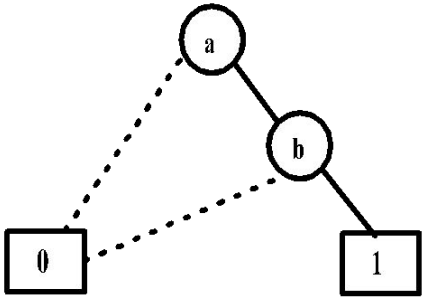
- However, construction of ROBDD using Shannon's expansion (of the output function) is more cumbersome compared to procedure of using Boolean operations on input ROBDDs. The reason is, determining Boolean function of a large circuit and then apply Shannon's expansion iteratively is a complex task.



(a) AND Gate



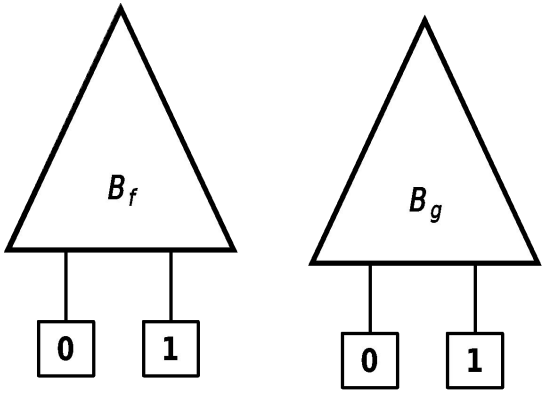
(b) ROBDD for "a" "b"



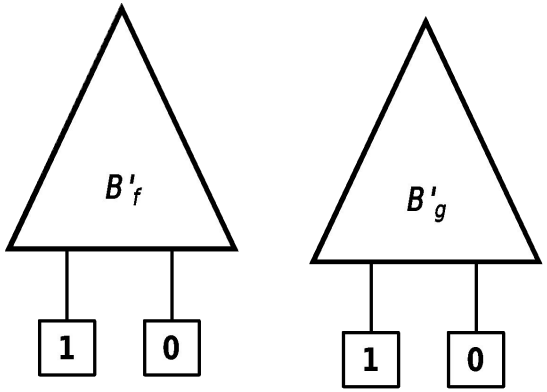
(c) ROBDD for out (ordering a<b)

Operations on ROBDD (Complement)

- ROBDDs B_f and B_g of two functions f and g (internal nodes are not shown).
- BDDs for the complement of f and g; it is very simple to get the BDD for the complement of a function, as we just need to reverse the values of leaf nodes.
- It may be noted that complement operation on ROBDD would result in a BDD that is ordered and reduced.



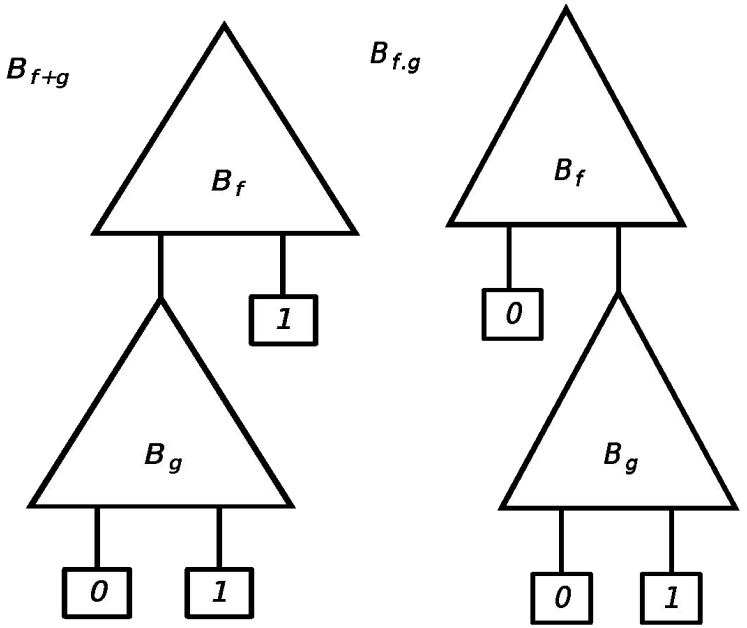
ROBDDs B_f and B_g of two functions f and g



Complement of ROBDDs B_f and B_g

Operations on ROBDD – AND/OR

- BDDs for the function ($f + g$) and ($f . g$).
- In case of "$f + g$" if f is 1, then we need not evaluate g and if f is "0" the final result depends only on g. This is reflected in the final BDD for "$f + g$".
- Similarly, in case of "$f.g$" if f is 0, then we need not evaluate g and if f is "1" the final result depends only on g.
- The way we are constructing the BDDs for OR and AND operation, it is clear that the resultant BDDs are not ordered or reduced. So, we need a method where the resultant BDD (after Boolean operations on ROBDDs) is also a ROBDD.



BDDs for the function ($f + g$) and ($f . g$)

Operations on ROBDD- *apply*

To perform the binary operation on two ROBDD's B_f and B_g, corresponding to the functions f and g respectively, we use the algorithm ***apply*(op, B_f , B_g).** The two ROBDDs B_f and B_g have compatible ordering.

The function ***apply*** is based on the Shannon's expansion for f and g:

$$f = \bar{x}.f[0/x] + x \cdot f[1/x]$$
$$g = \bar{x}.g[0/x] + x \cdot g[1/x]$$

From the Shannon's expansion of f and g :

$$f \ op \ g = \bar{x}.(f[0/x] \ op \ g[0/x]) + x.(f[1/x] \ op \ g[1/x])$$

This is used as a control structure of apply which proceeds from the roots of B_f and B_g downwards to construct nodes of the OBDD B_f op B_g. Let r_f be the root node of B_f and r_g *be* the root node of B_g.

Operations on ROBDD-*apply*

***apply*(op,B_f , B_g) Algorithm :**

1. If both r_f and r_g are terminal nodes with labels l_f and l_g, respectively compute the value l_f op l_g and the resulting OBDD is B_0 if the value is 0 and B_1 otherwise.

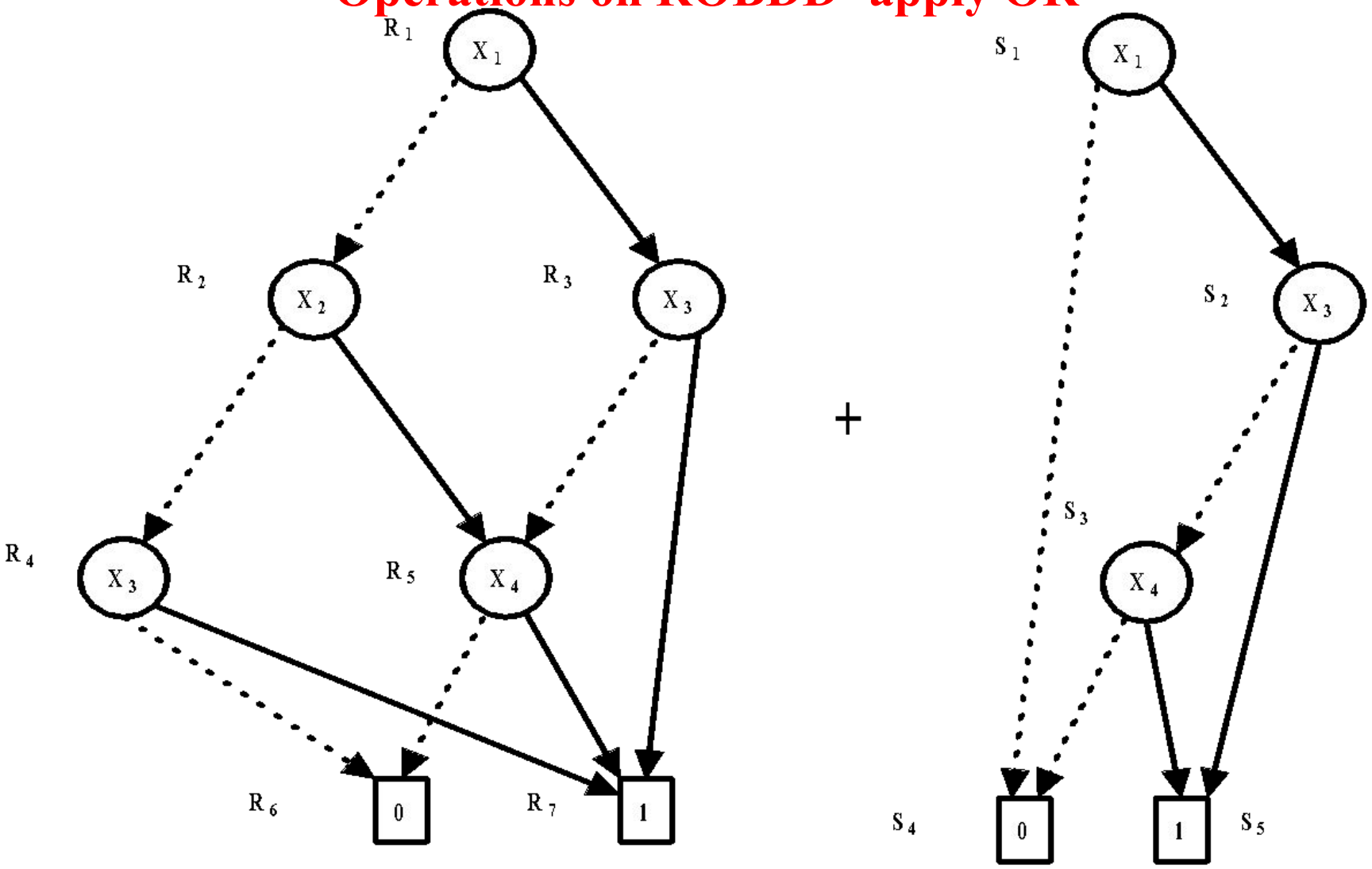
In the remaining cases, at least one of the root nodes is a non-terminal.

2. If both nodes are x_i-nodes (i.e., non-terminal of same order),
create an x_i-node n (called r_f,r_g) with a dashed line to ***apply*** (op, $lo(r_f)$, $lo(r_g)$)
and a solid line to ***apply***(op, $hi(r_f)$, $hi(r_g)$).

3. If r_f is an x_i-node, but r_g is a terminal node or an x_j-node with $j > i$,
create an x_i-node n (called r_f ,r_g) with a dashed line to ***apply***(op, $lo(r_f)$, r_g) and a solid line to ***apply***(op, $hi(r_f)$, r_g).

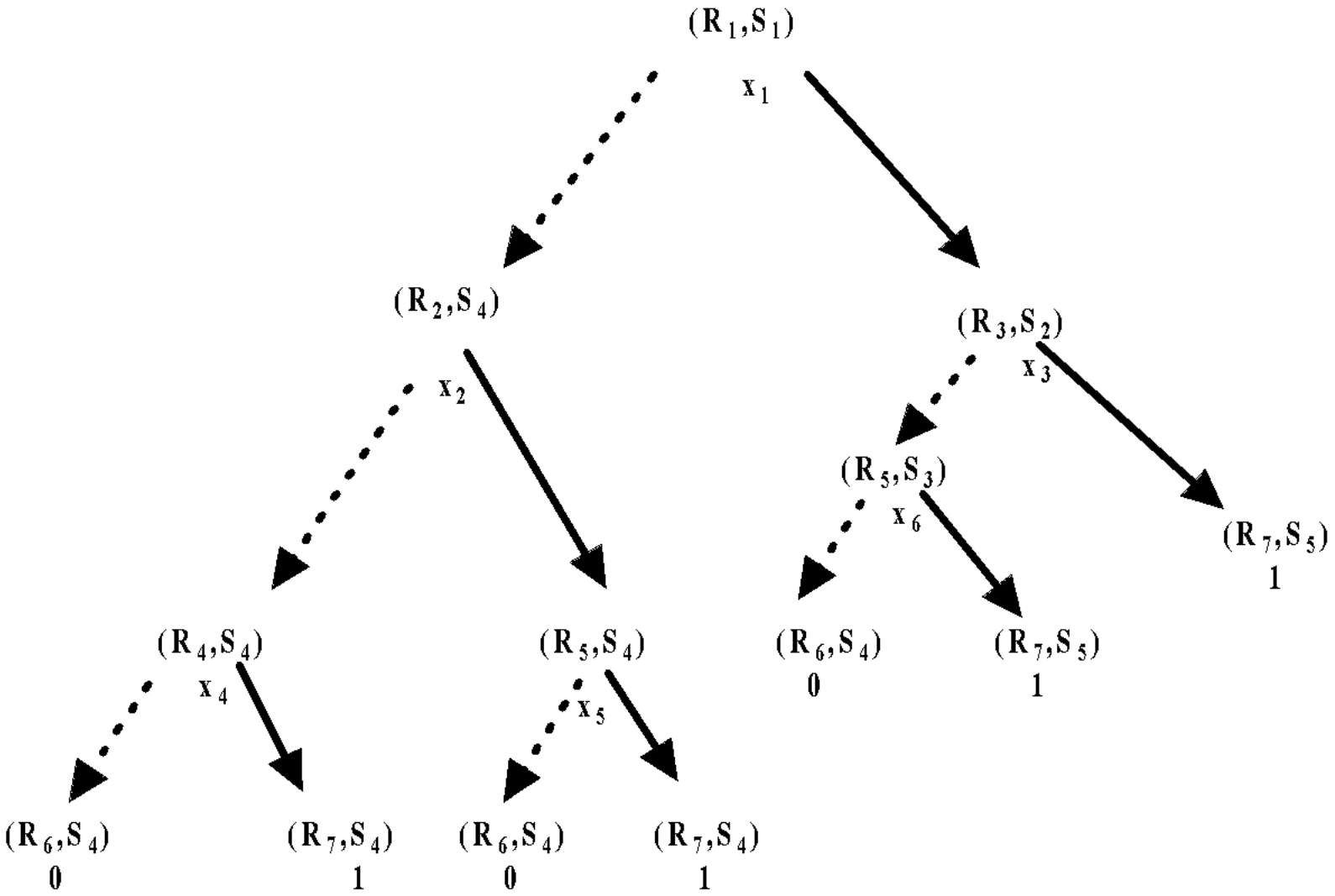
4. The case in which r_g is a non-terminal node, but r_f is a terminal or an x_j-node with $j > i$, is handled symmetrically to case 3.

Operations on ROBDD- apply OR



OR of two ROBDD's with ordering $[X_1,X_2,X_3,X_4]$

Operations on ROBDD--apply OR



Steps of the apply algorithm for the example

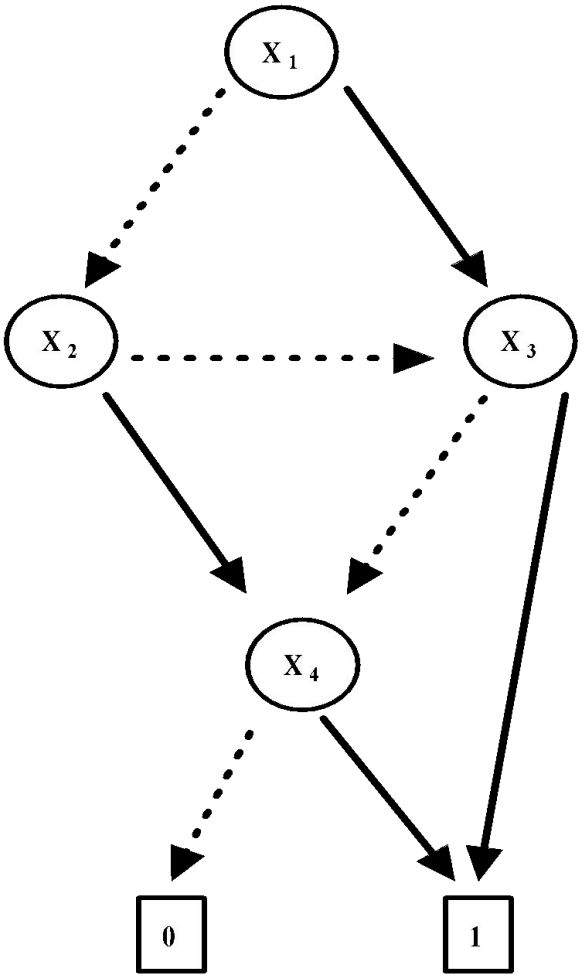
Operations on ROBDD--apply OR

- x_1 (R_1,S_1): According to Step-2 on R_1 and S_1--create an x_i-node n (called r_f,r_g =R_1,S_1).

- x_4 (R_4,S_4): According to Step-3 on x_2 (R_2,S_4) -- If r_f (R_2 in x_2) is an x_i-node, but r_g (S_4 in x_2) is a terminal node or an x_j-node with $j > i$, create an x_i-node n (called r_f,r_g=x_2) with a dashed line to ***apply***(*op, lo*(r_f), r_g)=(OR,R_4,S_4), which is node x_4.

- Terminal node (R_6,S_4): According to Step-3 on x_4 (R_4,S_4)-- If r_f (R_4 in x_4) is an x_i-node, but r_g (S_4 in x_4) is a terminal node or an x_j-node with $j > i$, create an x_i-node n (called r_f,r_g=x_4) with a dashed line to ***apply***(*op, lo*(r_f), r_g)=(OR,R_6,S_4), which is Terminal node (R_6,S_4). The value of R_6,S_4 is 0; *apply*(OR, R_6=0,S_4=0)=0.

Operations on ROBDD-apply OR



Final ROBDD

Operations on ROBDD-restrict

- The Boolean formula obtained by replacing all occurrences of x in f by 0 is denoted by $f[0/x]$. The formula $f[1/x]$ is defined similarly.

- The expressions $f[0/x]$ and $f[1/x]$ are called restriction of f

***restrict* (0/1, x, B_f)**
if $f[0/x]$ (i.e., *restrict(0, x, B_f))*
For each node n corresponding to x, remove n from OBDD and redirect incoming edges to *lo(n)*

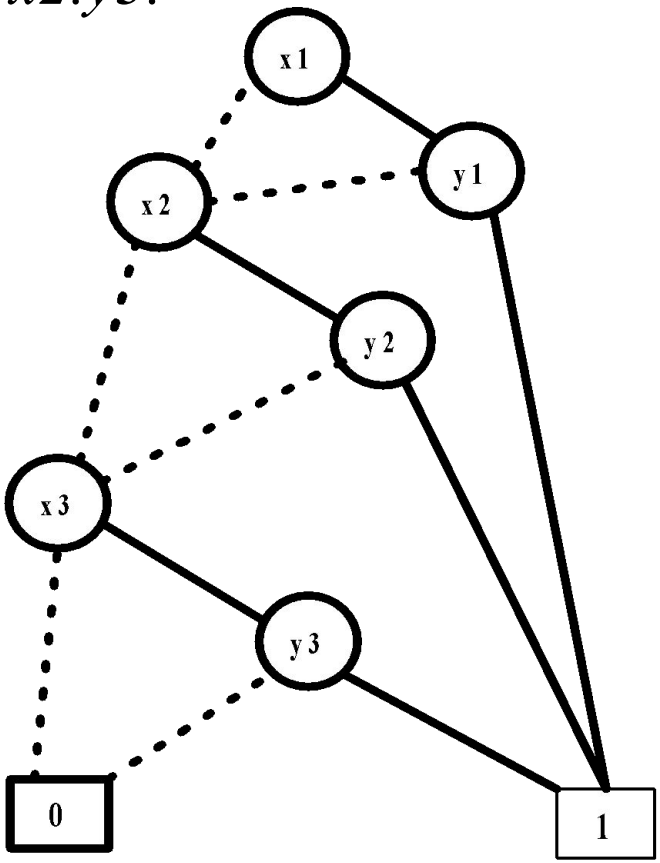
if $f[1/x]$ (i.e., *restrict(1, x, B_f))*
For each node n corresponding to x, remove n from OBDD and redirect incoming edges to *hi(n)*

Applying Reduce operation on the OBDD thus obtained to generate the ROBDD for the restrict function.

Operations on ROBDD-restrict (example)

$f = x1.y1 + x2.y3 + x3.y3$.

After applying ***restrict(0, x3, B_f)***, f becomes, $f[0/x3] = x1.y1 + x2.y3$.



ROBDD for $f = x1.y1 + x2.y3 + x3.y3$

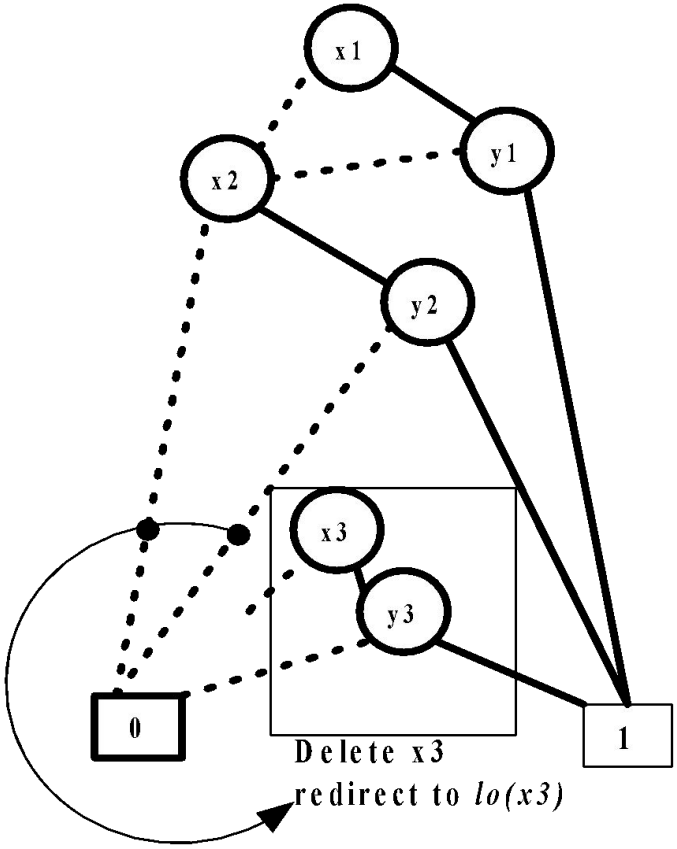


Illustration of Step1 of *restrict*

ADD, HDD based verification

Multi Terminal BDD

- Building on the success of BDDs, there have been several efforts to extend the concept to represent functions over Boolean variables, but having non-Boolean ranges, such as integers or real numbers.

- These functions arise in such applications as computing the state probabilities in a sequential circuit, using spectral methods for technology mapping, integer linear programming**,** and for verifying arithmetic circuits**.**

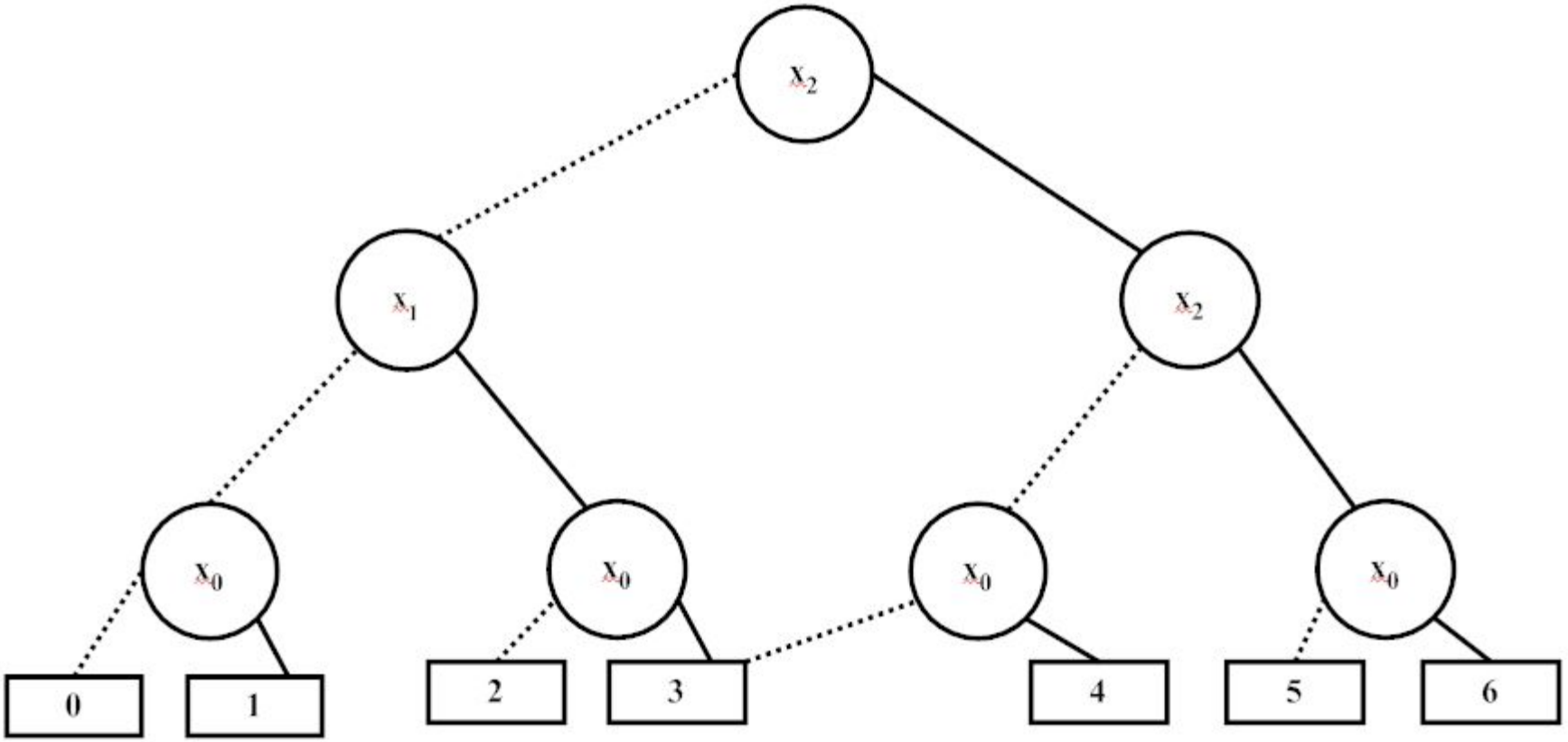
- Keeping the variables Boolean allows the use of a branching structure similar to BDDs. The challenge becomes finding a compact way to encode the numeric function values.

- One straightforward way to represent numeric-valued functions is use a decision graph like a BDD, but to allow arbitrary values on the terminal nodes.
  - "Multi-Terminal" BDD (MTDD), although they have also been called "Arithmetic Decision Diagrams" (ADDS)

MTBDD

MTBDD representation of the function $x_0 + 2\,x_1 + 3\,x_2$, i.e., the result of interpreting three bits as an unsigned binary number.

Evaluating an MTBDD for a given variable assignment is similar to evaluation in a BDD: we simply trace the unique path from the root to a terminal node determined by the variable values and return the terminal node value as the



MTBDD

- As this example illustrates, MTBDDs are very inefficient for representing functions yielding values over a large range.

- For the case of unsigned binary numbers of length n, there are 2^n possible values and hence the MTBDD representation must have an exponential number of terminal nodes.

Extensions

- Edge valued BDD (EVBDDs)
- Binary Moment Diagram (BMDs)
- High Level Decision Diagrams (HLDD)

High Level BDD

- BDDs, ADDs etc. are good in representing digital systems at logic gate level but, in case of complex systems, we often need to describe the systems at high levels, like behavioral, procedural or RTL.

- For this purpose High Level Decision Diagrams (HLDDs) were introduced to represent such systems at higher description levels.

- HLDDs have been used for high level and hierarchical test generation for complex digital circuits.

- The main advantage of using HLDDs is to generalize and extend the gate level methods and algorithms of fault simulation and test generation to higher abstraction levels.

- For this reason, the variables in the form of Boolean values are extended to Boolean vectors or integers and the Boolean functions are extended to data manipulation operations.

HLDD (High Level Decision Diagrams)

HLDD is a mathematical model used to represent digital systems at higher levels of abstraction. HLDD can be defined as a directed acyclic graph

$$GDD = (N, n_0, \Im, \Gamma, \gamma, X);$$

- where N is the finite set of *nodes* and $n_o \in N$ is the initial node.
- The set N is partitioned into two sets as $N = NT \cup T$, where NT is the set of non-terminal nodes and T is the set of terminal nodes.
- Each non-terminal node (say n_i) is associated with an expression(say exp_{ni}).
- The expression may be a control signal or a condition. Similarly, each terminal node is associated with an operation.

HLDD (High Level Decision Diagrams)

- HLDD is a mathematical model used to represent digital systems at higher levels of abstraction. HLDD can be defined as a directed acyclic graph

$$GDD = (N, n_0, \Im, \Gamma, \gamma, X);$$

- where N is the finite set of *nodes* and $n_o \in N$ is the initial node.
- The set N is partitioned into two sets as $N = NT \cup T$, where NT is the set of non-terminal nodes and T is the set of terminal nodes.
- Each non-terminal node (say n_i) is associated with an expression(say exp_{ni}).
- The expression may be a control signal or a condition. Similarly, each terminal node is associated with an operation.

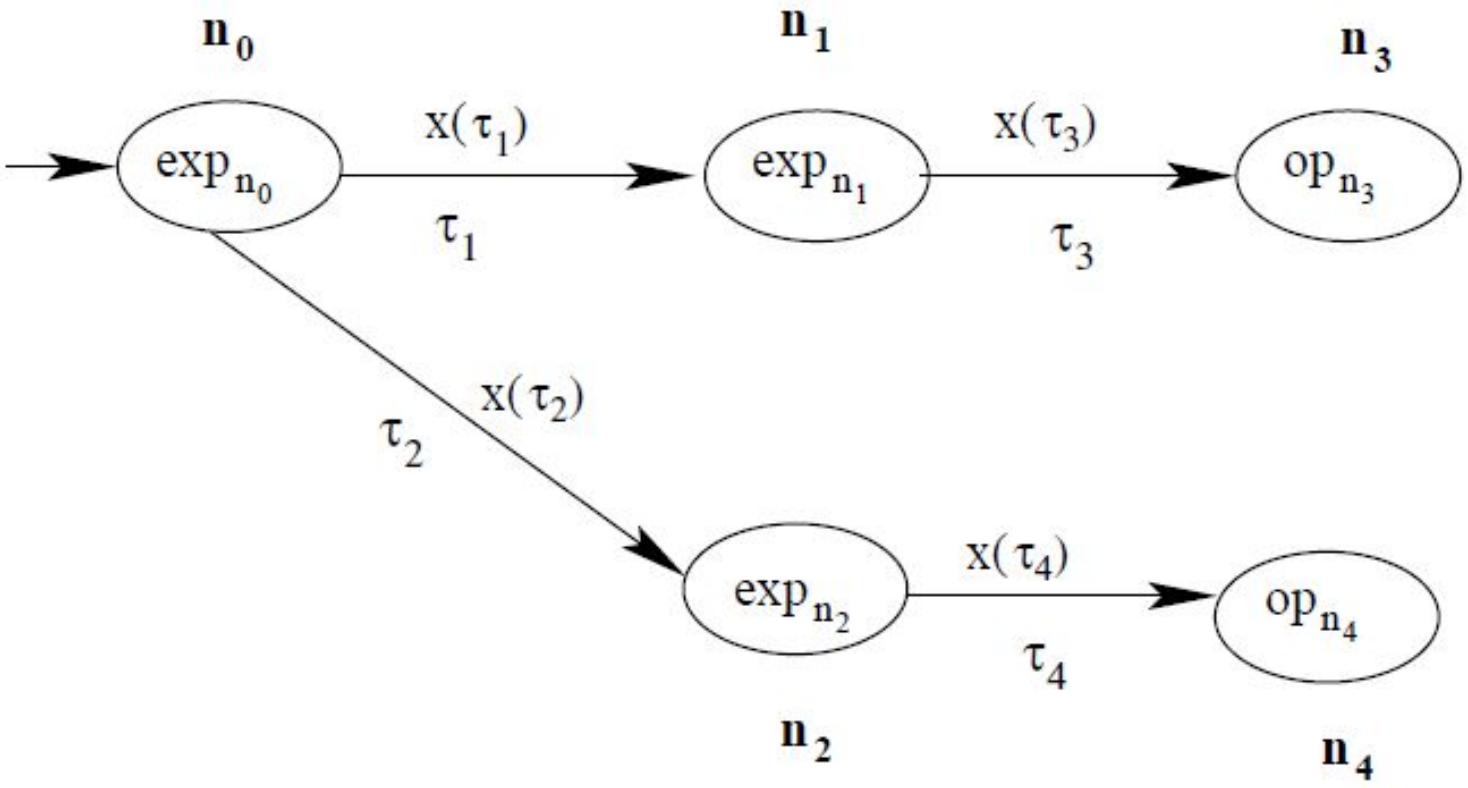
HLDD (High Level Decision Diagrams)

- HLDD is a mathematical model used to represent digital systems at higher levels of abstraction. HLDD can be defined as a directed acyclic graph

$$GDD = (N, n_0, \Im, \Gamma, \gamma, X);$$

- where N is the finite set of *nodes* and $n_o \in N$ is the initial node.
- The set N is partitioned into two sets as $N = NT \cup T$, where NT is the set of non-terminal nodes and T is the set of terminal nodes.
- Each non-terminal node (say n_i) is associated with an expression(say exp_{ni}).
- The expression may be a control signal or a condition. Similarly, each terminal node is associated with an operation.

HLDD (High Level Decision Diagrams)



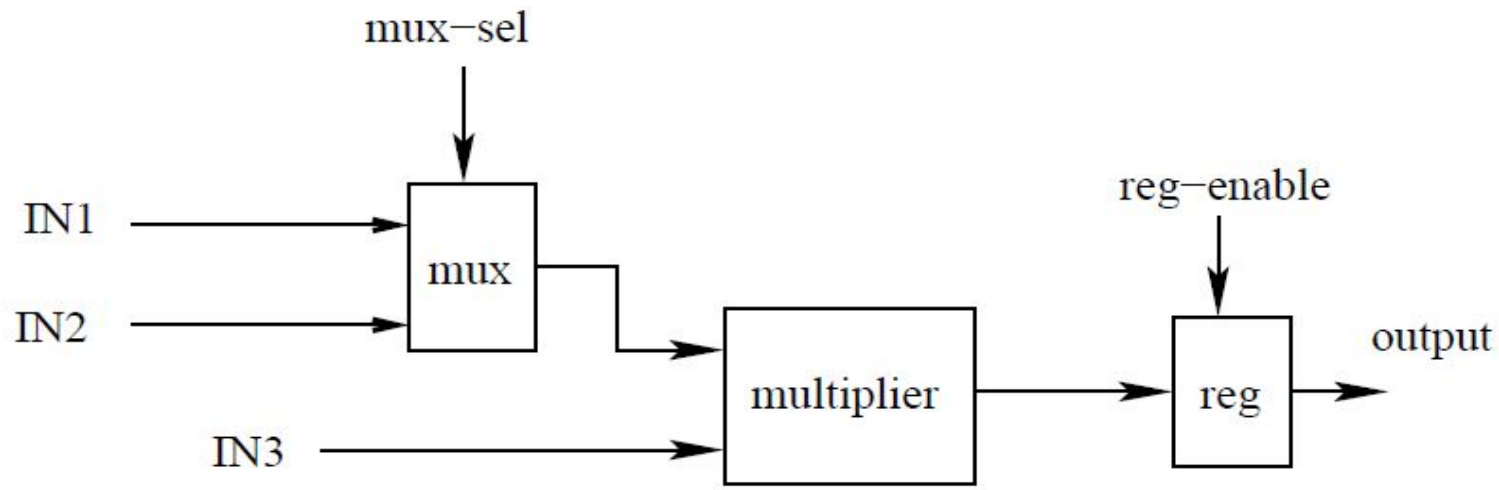
HLDD (High Level Decision Diagrams)

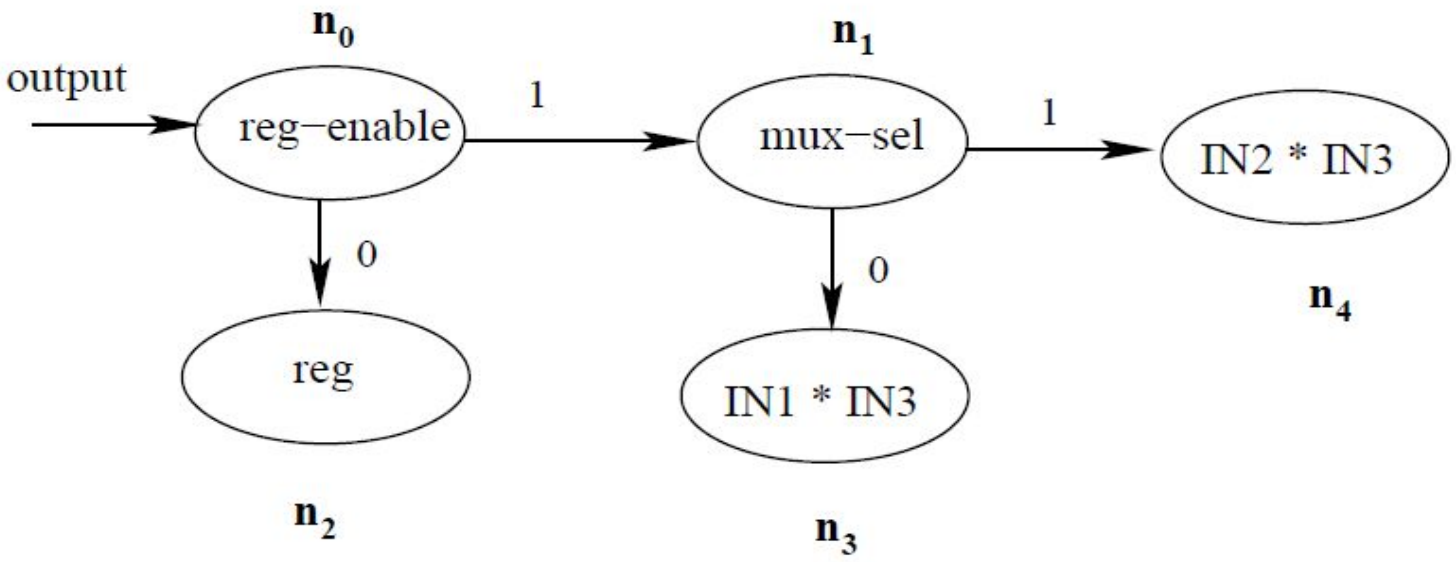
HLDD is a mathematical model used to represent digital systems at higher levels of abstraction. HLDD can be defined as a directed acyclic graph

$$GDD = (N, n_0, \Im, \Gamma, \gamma, X);$$

- where N is the finite set of *nodes* and $n_o \in N$ is the initial node.
- The set N is partitioned into two sets as $N = NT \cup T$, where NT is the set of non-terminal nodes and T is the set of terminal nodes.
- Each non-terminal node (say n_i) is associated with an expression(say exp_{ni}).
- The expression may be a control signal or a condition. Similarly, each terminal node is associated with an operation.

HLDD (High Level Decision Diagrams)-Example





HLDD (High Level Decision Diagrams) -- Example

Example of a data path of an RTL circuit.

- The circuit performs multiplication of two input numbers among three numbers and stores the product in the register (reg) and finally outputs the product.

- The first input i.e., IN3 is directly applied to the multiplier whereas, the second input between IN1 and IN2 is selected by the multiplexer (mux).

- When the multiplexer selection line, i.e., mux-sel is 0(1), then IN1(IN2) is selected by the multiplexer and applied to the multiplier. The multiplication is performed (either IN1 $*$ IN3 or IN2 $*$ IN3) and the product is stored in the register when the register enable signal, i.e., reg-enable is high.

HLDD (High Level Decision Diagrams) -- Example

- The initial node n0 of the HLDD represents the reg and is associated with the expression reg-enable, i.e., enable signal of reg.

- node n1 represents the mux and is associated with the expression mux-sel, i.e., selection line of mux. The content of the register remains unchanged as long as reg-enable=0 ; this is modeled at terminal node n2, labeled by constant assignment operation, which assigns the previous value of the register as output.

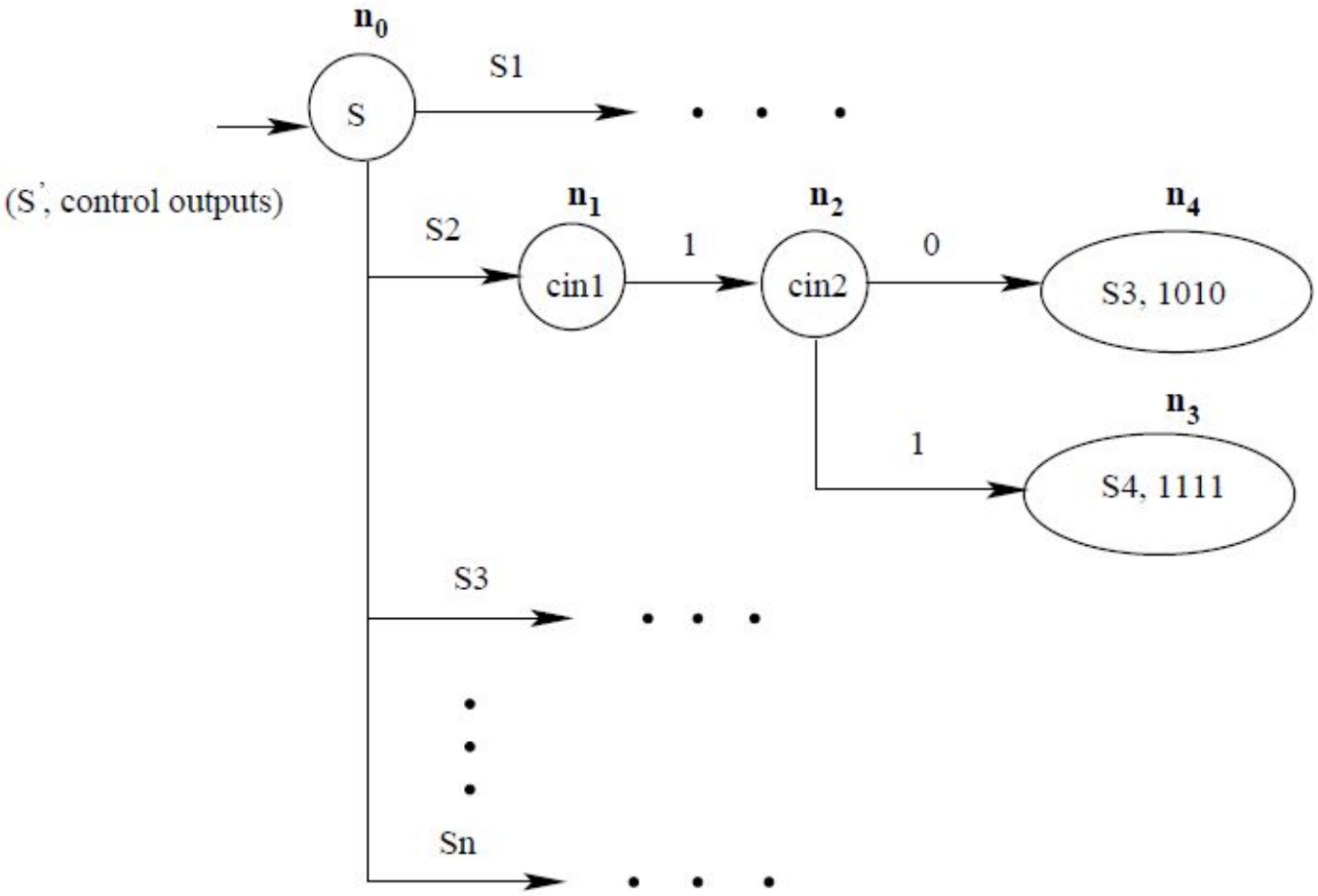
- Similarly, two multiplication operations IN1 * IN3 and IN2 * IN3 are carried out at the terminal nodes n3 and n4, respectively.

- Thus, in the data path HLDD model, the non-terminal nodes are labeled by some control or selection expressions and the terminal nodes are labeled by some operations

HLDD (High Level Decision Diagrams)—Example Control Path

Present state	Control inputs (cin1, cin2)	Next state	Control outputs
•	•	•	•
S2	(1 0)	S3	(1 0 1 0)
S2	(1 1)	S4	(1 1 1 1)
•	•	•	•
•	•	•	•

HLDD (High Level Decision Diagrams)—Example Control path



HLDD (High Level Decision Diagrams)—Example
Control path

- In the control HLDD model, the non-terminal nodes correspond to the current *state status* and FSM *input control signals* (*conditions*) whereas, terminal nodes hold a vector which includes *values of next state* along with the values of FSM *output control signals*.

- Here, the FSM state table shows behavior of the circuit when the present state is *S*2. The initial node n_0 in the HLDD model represents present state and is associated with current state status *S*.

- The non-terminal nodes n_1 and n_2 represent FSM inputs and are associated with control signals *cin*1 and *cin*2, respectively.

- The terminal odes *n*3 and *n*4 represent next state and FSM outputs when the values of <*cin*1, *cin*2> is 10 and 11, respectively.

Thank You